UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA



Carrera: Ingeniería en Sistemas de Información y Ciencias de la Computación
Catedrático: Ing. Marco Tulio Valdez
Curso: Algoritmos

PROYECTO FINAL

Integrantes :	**Carné:**
Herbert Mauricio Lucero Lemus	0907-21-11994
Cristian René Hernández Ucelo	0907-26-18257
Erick Abdiel Galvez Orellana	0907-26-18245

24 de octubre del 2026

Informe Final: Sistema de Inventario en Python

Introducción
Este informe explica el proyecto final del curso de Algoritmos de la Universidad Mariano Gálvez de Guatemala, Centro Jalapa. El proyecto consistió en hacer un programa en Python para llevar el control de un inventario: los productos que se venden, los clientes y los pedidos que ellos hacen. En las empresas es muy común que este control se lleve a mano o en hojas sueltas, y eso causa errores. Por eso el programa guarda toda la información en un archivo de Excel llamado Inventario.xlsx , calcula solo los totales de las ventas y puede enviar por correo una cotización en Word y una copia de seguridad. El programa tiene una ventana con pestañas, hecha con la biblioteca Tkinter que vimos en clase, para que cualquier persona lo pueda usar sin escribir comandos. También se hicieron pruebas unitarias para comprobar que las partes más importantes funcionan bien.

Integrantes:
Herbert Mauricio Lucero Lemus
Cristian René Hernández Ucelo
Erick Abdiel Galvez Orellana

Funcionamiento del programa
Al ejecutar la app.py se abre una ventana con cinco pestañas . La primera vez que se abre, el programa crea el archivo Inventario.xlsx con tres hojas: productos, clientes y pedidos. Cada vez que se agrega, edita o elimina algo, el cambio se guarda en ese archivo, así que al cerrar el programa y abrirlo otra vez los datos siguen ahí.

Productos.
Se escribe el nombre, el precio y la existencia, y se agrega con un botón. Si se hace clic en una fila de la tabla, sus datos pasan a las casillas para poder editarlos o eliminarlos. También hay un botón para enviar la cotización: se escoge el producto, el programa pide el nombre y el correo del cliente, crea el documento cotización.docx y lo envía como adjunto.

Clientes.
Funciona igual que los productos, pero con nombre, NIT y dirección. No deja guardar dos clientes con el mismo nombre ni con el mismo NIT.

Pedidos.
Se elige un cliente y un producto de dos listas desplegables y se escribe la cantidad. El programa calcula el valor del pedido (precio por cantidad) y le resta esa cantidad a la existencia del producto. Si no alcanza la existencia, no deja hacer el pedido. Cuando se elimina un pedido, la cantidad vuelve al producto.

Informes.
Hay dos botones: el total de venta por cliente (con un total general) y el total de ventas por producto, con las unidades vendidas y el dinero.

Varios.
Tiene el botón de copia de seguridad, que envía el archivo Inventario.xlsx completo por correo al coordinador del grupo. En todas las pestañas, si el usuario escribe un dato incorrecto (por ejemplo letras en el precio o una cantidad negativa), aparece un mensaje de error en lugar de que el programa se cierre.

Explicación del código
El proyecto está dividido en varios archivos para que esté más ordenado. La idea principal es que app.py solo se encarga de la ventana y datos.py se encarga de las reglas, del Excel, de Word y del correo.

Archivo	Para qué sirve
`app.py`	Crea la ventana, las pestañas, las tablas y los botones, y llama a las funciones de `datos.py`
`datos.py`	Guarda las funciones que leen y escriben el Excel, validan datos, calculan pedidos e informes, crean el Word y envían el correo
`secretos.py`	Guarda el correo, la contraseña de aplicación y el correo del coordinador. No se sube a GitHub
`secretos_ejemplo.py`	Plantilla sin datos reales para que otra persona sepa qué debe llenar
`test_datos.py` y `test_logica.py`	Pruebas unitarias del programa (35 en total)
`.gitignore`	Le dice a Git qué archivos no debe subir

Funciones de datos.py
Pruebas unitarias del programa (35 en total) Le dice a Git qué archivos no debe subir Todo el programa sigue una misma idea: leer todo el Excel, cambiar la lista en memoria y volver a guardar todo.

Función	Qué hace
`crear_libro()`	Crea `Inventario.xlsx` con las tres hojas y sus encabezados, solo si el archivo no existe
`leer(hoja)`	Lee las filas de una hoja (sin el encabezado) y las devuelve como una lista de listas
`guardar(hoja, filas)`	Borra las filas de la hoja y escribe las que recibe
`validar_producto(...)`	Revisa que el nombre no esté vacío, que el precio sea número y la existencia un entero, y que no sean negativos
`registrar_pedido(...)`	Valida el pedido, comprueba la existencia, la descuenta, calcula el valor y guarda el pedido
`anular_pedido(posicion)`	Elimina un pedido y le devuelve la cantidad al producto
`calcular_total_por_cliente()`	Suma lo vendido a cada cliente usando un diccionario
`calcular_total_por_producto()`	Suma unidades y dinero por producto usando un diccionario con tuplas
`crear_cotizacion(...)`	Crea `cotización.docx` reemplazando los datos que van entre arrobas
`enviar_correo(...)`	Arma el mensaje con su adjunto y lo envía por Gmail

Cuando algo está mal, estas funciones usan raise ValueError(...) para avisar del error. La ventana lo atrapa con try y except ValueError as error y lo muestra en un cuadro de mensaje.

Partes de app.py
Para la ventana usamos tk.Tk() y ttk.Notebook para las pestañas. En cada pestaña hay Entry para escribir, Combobox para las listas desplegables, Button con command= para las acciones y Treeview para mostrar las tablas. Los mensajes salen con messagebox. Las funciones que se llaman al hacer clic (agregar_producto, editar_cliente, eliminar_pedido, etc.) leen las casillas, llaman a datos.py y vuelven a llenar la tabla.

Bibliotecas usadas

- **Tkinter:** la ventana. Es la que vimos en clase.
- **openpyxl:** leer y guardar el archivo de Excel.
- **python-docx:** crear el documento de Word.
- **smtplib y email.message:** enviar el correo con Gmail por el puerto 587 con starttls.
- **os:** armar la ruta del archivo para que el Excel siempre quede junto al programa.

- **unittest, mock y tempfile: l**as pruebas, para no tocar el Excel real ni enviar correos de verdad.

Pruebas unitarias
Hicimos 35 pruebas que revisan el Excel, el Word, el correo (con un servidor falso), las validaciones, los pedidos y los informes.

Uso de inteligencia artificial
Como se indicó en las instrucciones del proyecto, utilizamos herramientas de IA únicamente como apoyo para resolver dudas puntuales de sintaxis y librerías que no conocíamos a fondo.

1. Herramienta utilizada
Se usó Claude, de la empresa Anthropic.

prompt que se usaron:

Prompt 1. Aprender Tkinter y la interfaz gráfica

Estoy trabajando en una aplicación de Python llamada Sistema de Inventario y quiero entender cómo funciona la interfaz gráfica que hice con Tkinter.

Prompt 2. Aprender a guardar y leer información de Excel

Quiero aprender cómo funciona la biblioteca openpyxl que utilizo en mi aplicación de inventario.

Explícame paso a paso cómo funcionan `Workbook()`, `load_workbook()`, `create_sheet()`, `iter_rows()`, `delete_rows()`, `append()`, `save()` y `close()`.

Prompt 3. Crear documentos Word desde Python

Quiero aprender cómo funciona la biblioteca python-docx que utilizo en mi sistema de inventario para generar cotizaciones.

Prompt 4. Enviar correos electrónicos con archivos adjuntos

Quiero entender cómo funciona el envío de correos electrónicos en mi aplicación de inventario.

Explícame paso a paso qué hacen las bibliotecas smtplib y email.message, y cómo funcionan EmailMessage(), set_content(), add_attachment(), SMTP(), starttls(), login() y send_message().

Prompt 5. Manejar errores con try y except

Explícame cuándo se ejecuta el bloque `try`, qué sucede cuando ocurre un error y cómo funciona

Enséñame qué ocurriría si el usuario escribe letras en lugar de números y cómo evitar que la aplicación se cierre inesperadamente.

Aporte Propio del Equipo en la Aplicación

Aunque consultamos ejemplos para entender ciertas librerías, todo el diseño y ensamblado del proyecto fue realizado por nosotros:

1. Diseñamos la lógica del menú y la navegación entre las pantallas de la interfaz gráfica.
2. Creamos la estructura del archivo Inventario.xlsx con sus 3 hojas e hicimos que las fórmulas de importes cuadraran correctamente.
3. Escribimos las validaciones para evitar errores (por ejemplo, comprobar que no se ingresen letras en el precio o cantidad).
4. Pruebas e integración final del código para asegurarnos de que funcionara completo al ejecutar app.py.

diagramas usados para entender el funcionamiento del programa y el orden de ejecuciòn

Diagramas de Flujo: Sistema de Inventario

Diagrama 1: Agregar un producto

Un producto solo se guarda si el nombre no está vacío, el precio y la existencia son números válidos y el nombre no se repite; en cualquier otro caso se muestra un mensaje de error.

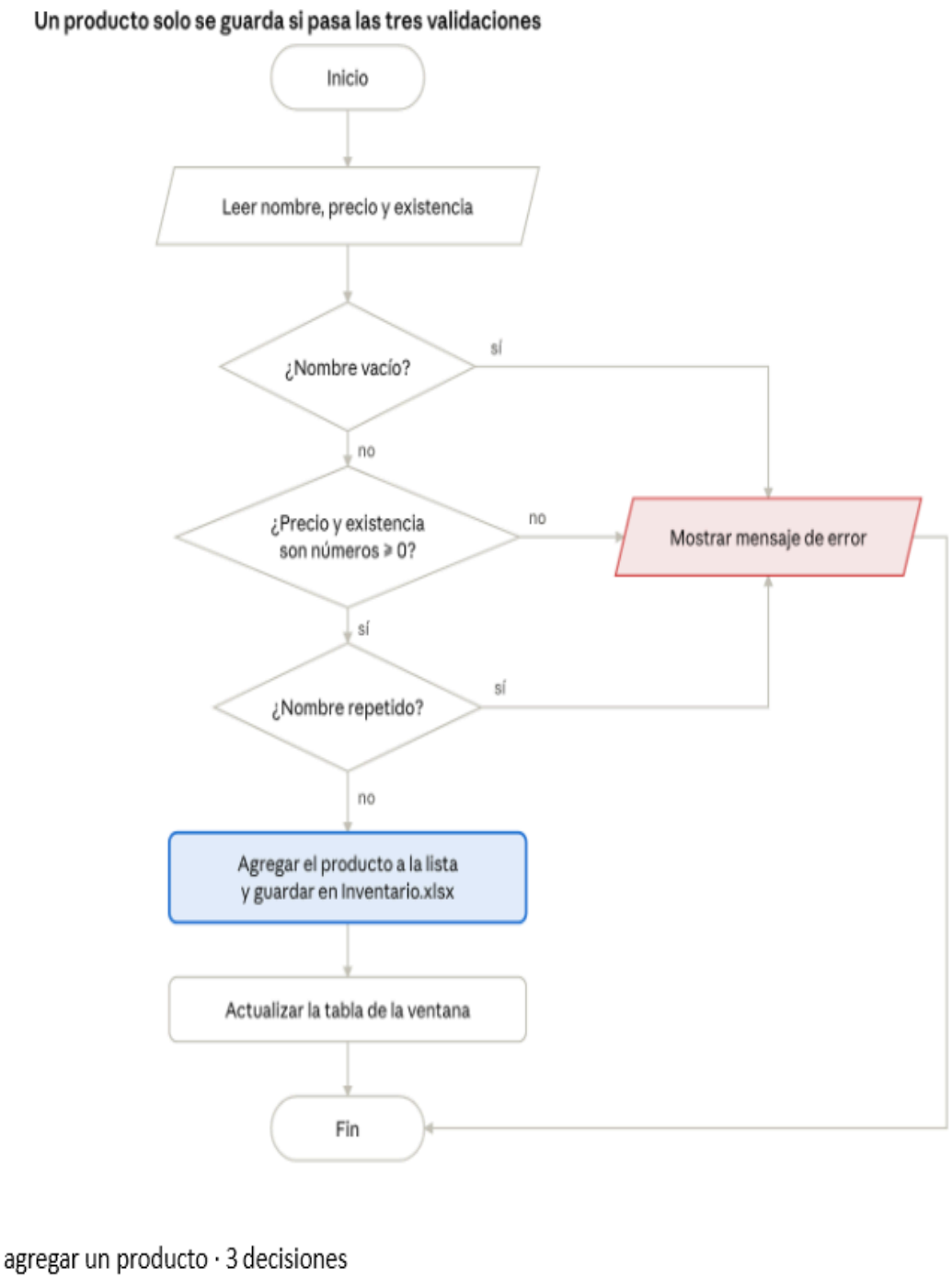


Diagrama 2: Registrar un pedido

El pedido se guarda solo si se eligió cliente y producto, la cantidad es un entero mayor que cero, ambos existen y alcanza la existencia; entonces se descuenta del inventario y se calcula el valor.

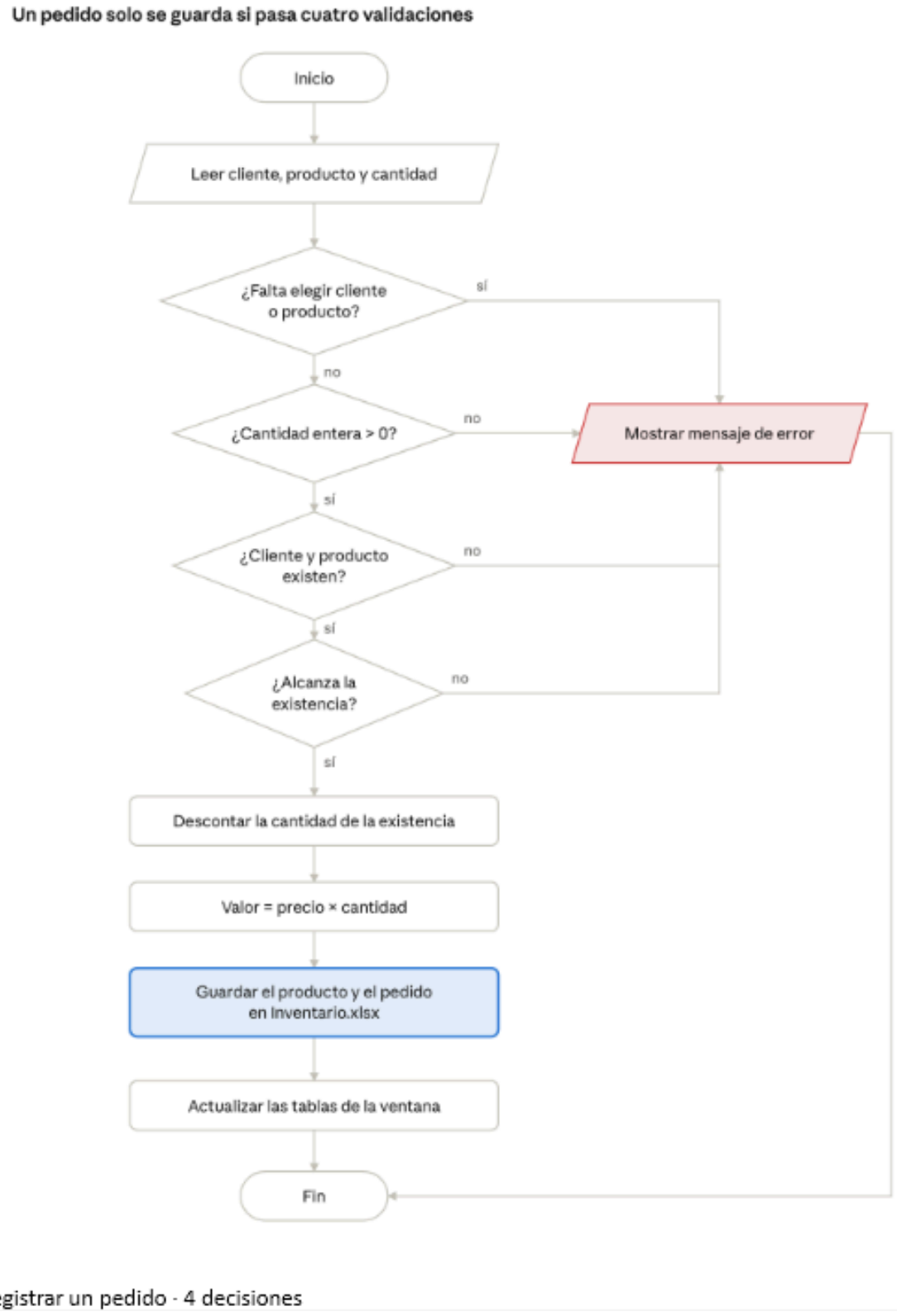


Diagrama 3: Eliminar un pedido

Si hay un pedido seleccionado y el usuario confirma, el pedido sale de la lista y su cantidad se suma de nuevo a la existencia del producto; si no confirma, no cambia nada.

Al eliminar un pedido, su cantidad vuelve a la existencia del producto

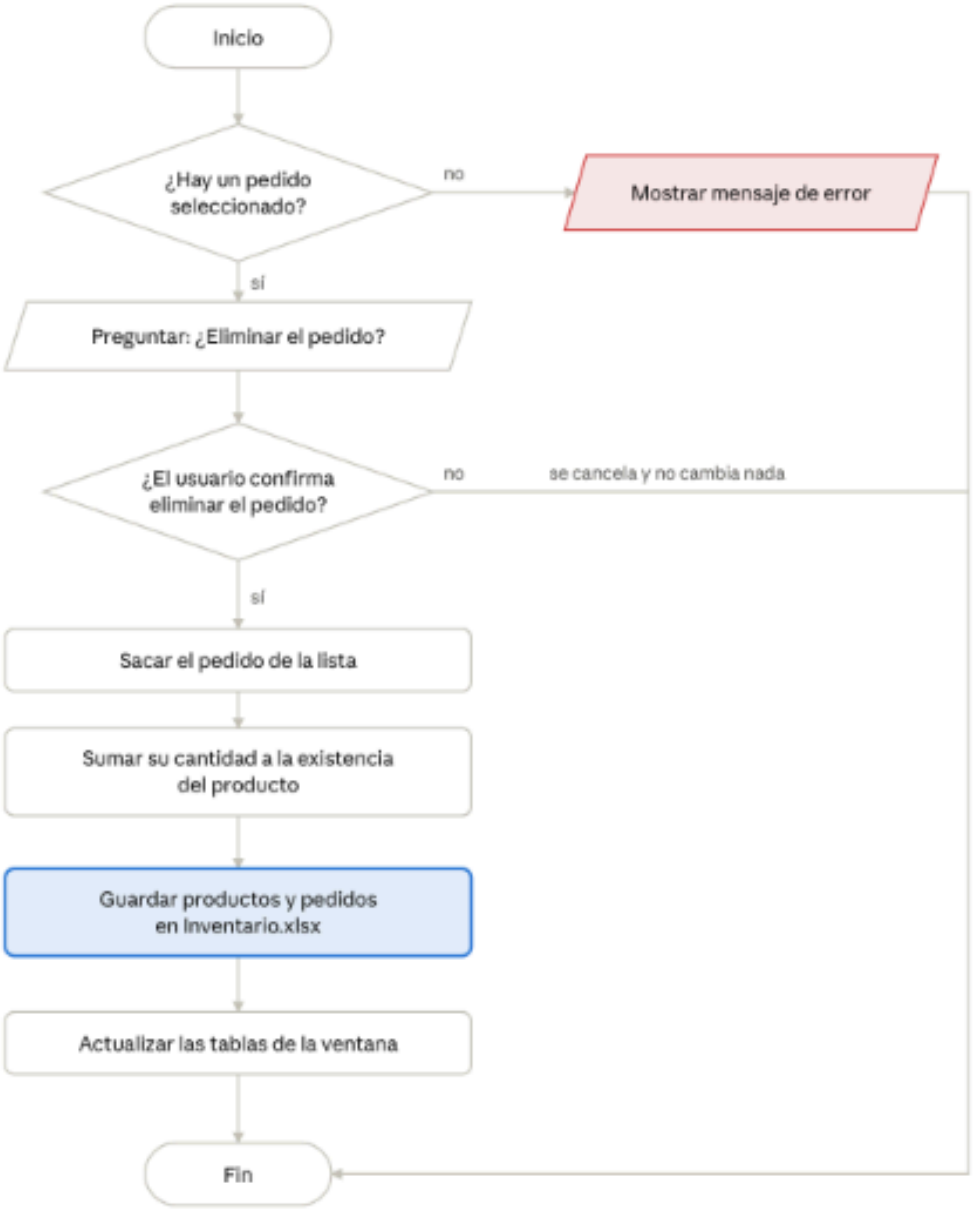


Diagrama 4: Enviar una cotización por correo
El programa pide el nombre y el correo del cliente, crea el documento de Word y lo envía por Gmail; si el usuario cancela, no pasa nada, y si algo falla (sin producto, correo inválido o error de conexión) muestra un mensaje de error.

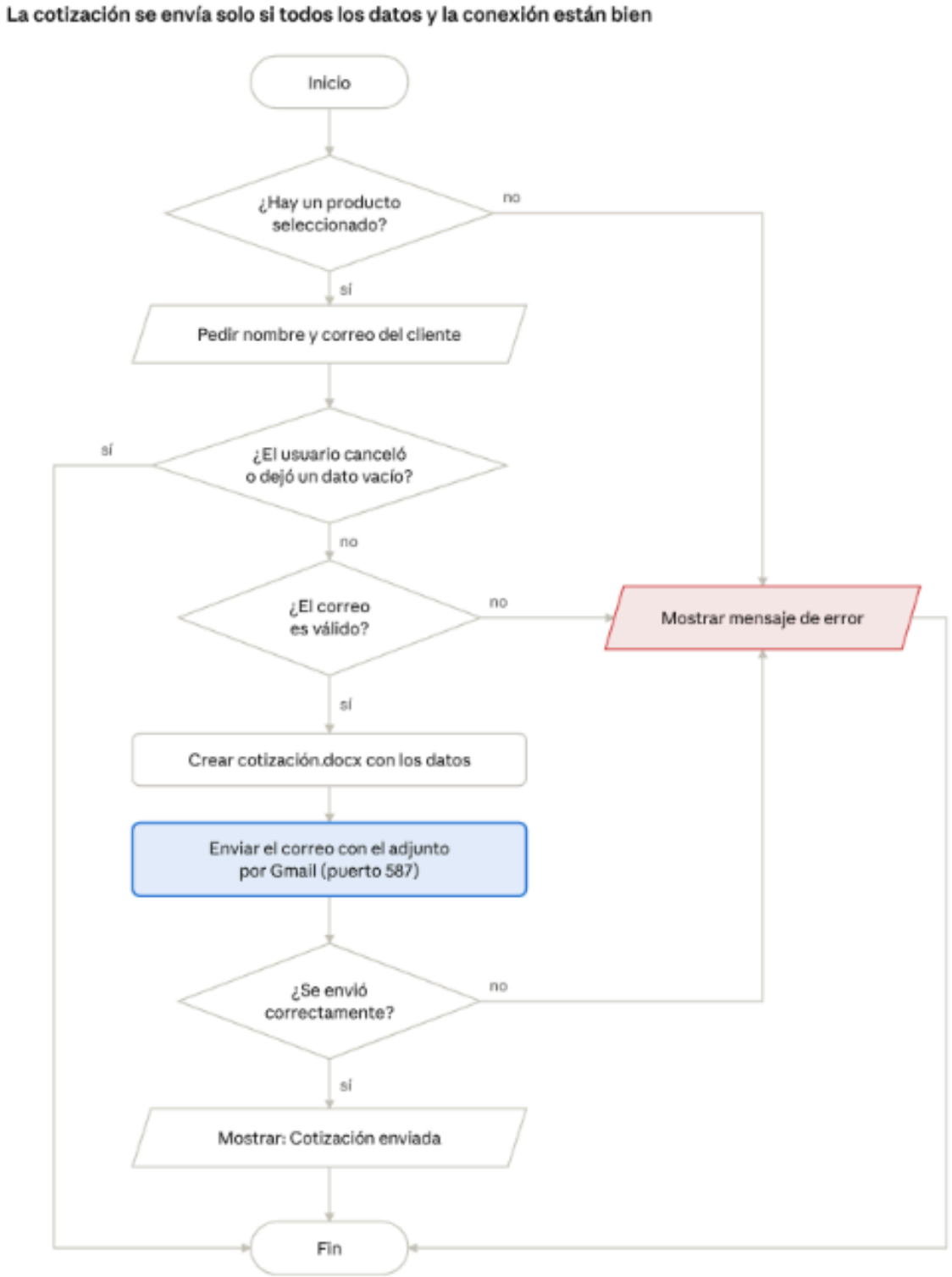
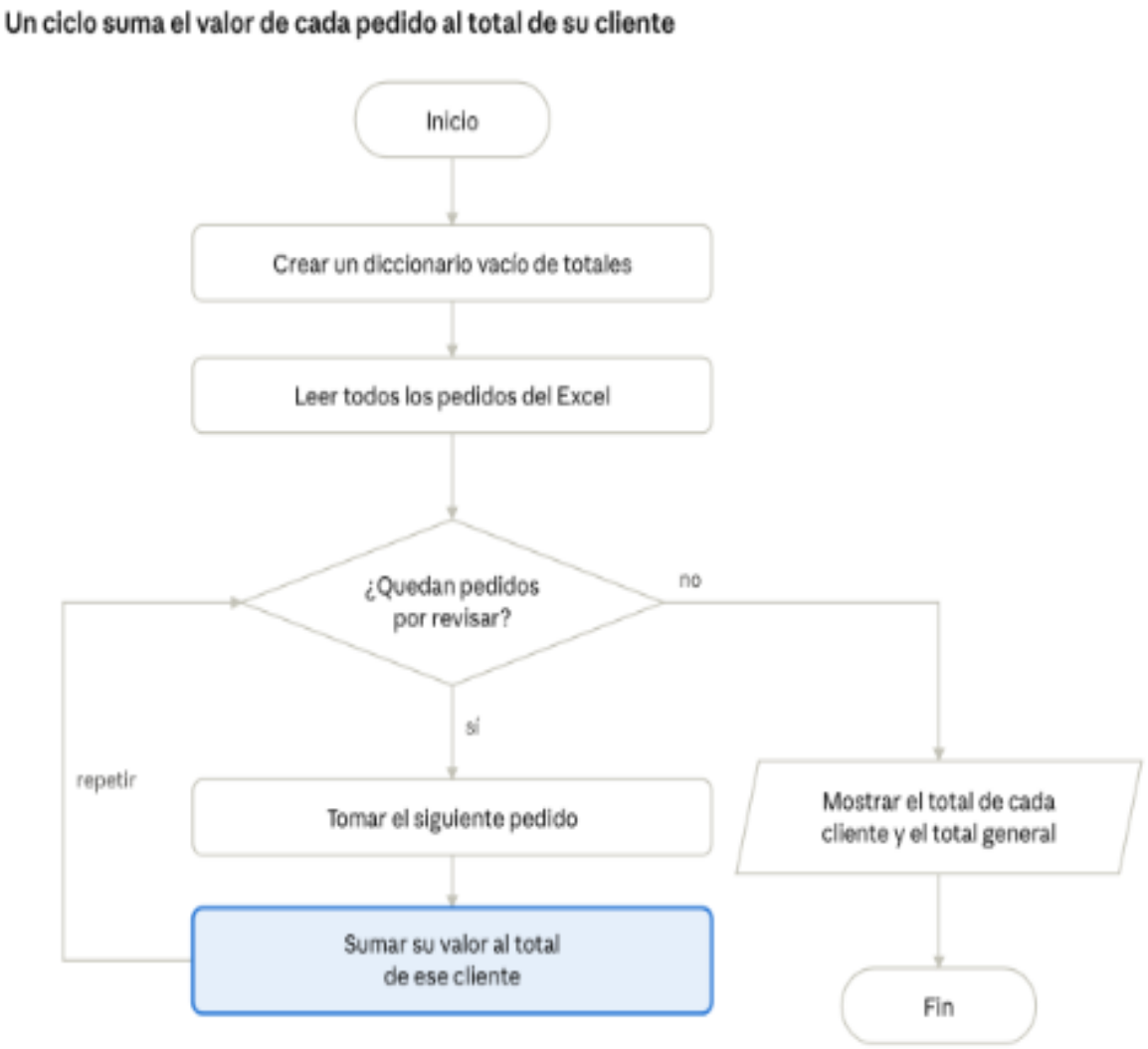
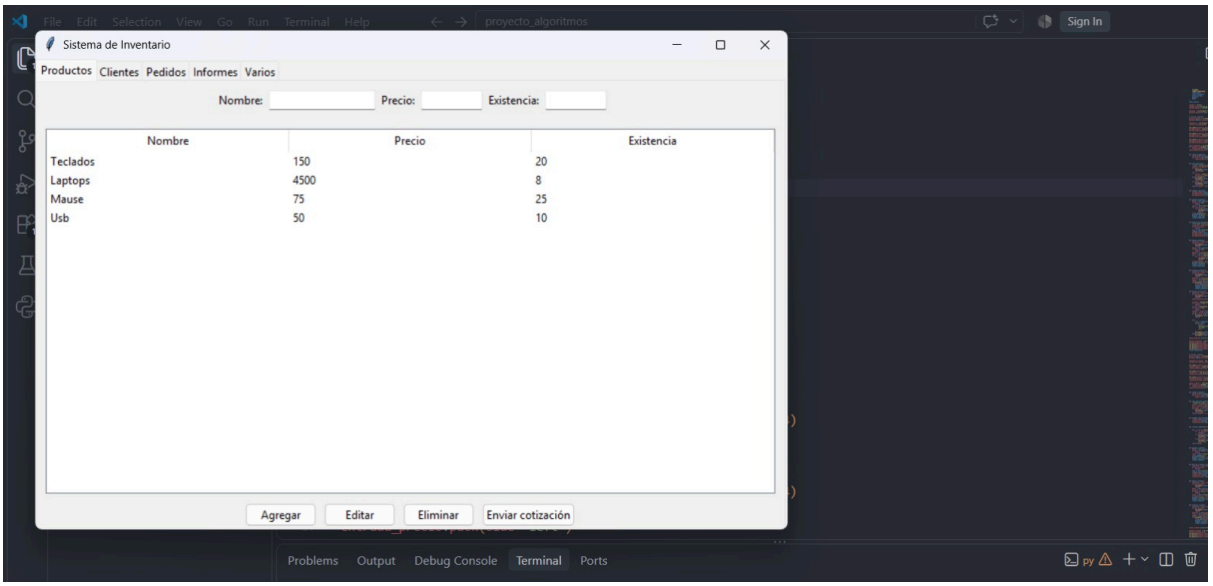


Diagrama 5: Total de ventas por cliente
El informe recorre los pedidos uno por uno con un ciclo y va sumando cada valor al total de su cliente en un diccionario; cuando ya no quedan pedidos, muestra los totales.



evidencia de la página terminada



Conclusión El programa cumple con las funciones principales que pedía el enunciado: se pueden manejar productos, clientes y pedidos, se guardan los datos en un archivo de Excel, se sacan informes de ventas y se pueden enviar por correo la cotización en Word y la copia de seguridad. Todo esto se usa desde una ventana con pestañas, por lo que no hace falta saber programar para usarlo.

Desarrollar este programa nos permitió comprender cómo los algoritmos que aprendemos en clase se convierten en herramientas reales y útiles para una empresa. Logramos conectar una interfaz visual con un archivo de Excel para guardar datos de forma permanente, además de automatizar tareas como el envío de correos y la edición de documentos de Word.

APRENDIZAJE OBTENIDO:

El mayor aprendizaje fue perder el miedo a trabajar con librerías externas de Python que no habíamos usado antes (como openpyxl, python-docx y smtplib). Aprendimos la importancia de estructurar bien el código en funciones, validar lo que escribe el usuario para que el programa no se trabe y trabajar colaborativamente dividiéndonos tareas dentro del equipo de trabajo.

Con este proyecto aprendimos a juntar muchas cosas que vimos por separado en clase: listas, diccionarios, tuplas, funciones, manejo de archivos y Tkinter. También aprendimos cosas nuevas, como leer y guardar archivos de Excel con openpyxl , crear un documento de Word con python-docx y enviar correos con smtplib . Lo que más nos enseñó fueron los errores., que una función repetida en dos archivos puede dar un NameError , y que un error de conexión no siempre es culpa de la red. Aprendimos a probar cada parte antes de seguir y a leer el mensaje de error completo. También aprendimos la importancia de las pruebas unitarias: sirven para comprobar que el programa sigue funcionando después de hacer

cambios. Por último, entendimos que usar IA ayuda a avanzar más rápido, pero que lo importante es entender lo que hace cada parte del código para poder explicarlo y modificarlo nosotros mismo.